

Given as input (i) a rule  $r$ , (ii) a desired number  $n$  of examples, (iii) an integer  $m$  to indicate the maximum number of facts given as a context, and (iv) a pool of values for each type involved in  $r$ 's predicate  $poools$ , Algorithm 1 outputs a dataset  $D$  of generated examples.

---

**Algorithm 1: Generate Synthetic Data**

---

```

Input: rule  $r$ ;      //  $child(a,b) \rightarrow parent(b,a)$ 
          $n$ ;           // # of examples
          $m$ ;           // max # of facts
          $poools$ ;      // pools of names

Output: Generated Dataset  $D$ 

1  $D = \{\}, i = 1$ ;           // initialize
2 while  $i \leq ceiling(n/8)$  do
3    $F = GenFacts(r, m, poools)$ ;
   //  $child(Eve, Bob), parent(Amy, Sam)$ 
4    $O = LPMLN(r, F)$ ;      // reasoner output
5    $h_1 = f \in F$ ;         //  $child(Eve, Bob)$ 
6    $h_2 = Alter(f)$ ;       //  $negchild(Eve, Bob)$ 
7    $h_3 = r(F)$ ;          //  $parent(Bob, Eve)$ 
8    $h_4 = Alter(r(F))$ ;    //  $parent(Eve, Bob)$ 
9    $h_5 = pos.f_l \notin F$ ; //  $child(Joe, Garry)$ 
10   $h_6 = \neg h_5$ ;         //  $negchild(Joe, Garry)$ 
11   $h_7 = f_r \notin O$ ;    //  $parent(Alice, Joe)$ 
12   $h_8 = \neg h_7$ ;       //  $negparent(Alice, Joe)$ 
13   $D.add(h_{1-8})$ ;
14   $i \leftarrow i + 1$ ;

15 Function  $GenFacts(r, m, poools)$  :
16    $F = GetRandomFacts(r, poools, m)$ ;
17    $F.add(GetRuleFacts(r, poools))$ ;
18   return  $F$ 

19 Function  $Alter(p(s, o))$  :
20   if  $p$  is symmetric then return  $\neg p(s, o)$ ;
21   if  $random() > 0.5$  then return  $\neg p(s, o)$ ;
22   else return  $p(o, s)$ ;

```

---

We start at line 3 by generating facts, such as  $child(Eve, Bob)$ , using the function  $GenFacts$  (lines 15–18), which takes as input a rule  $r$ , the maximum number of facts  $m$  to generate, and the  $poools$ . A random integer less than  $m$  sets the number of facts in the current context. The generated facts  $F$  have predicates from the body of  $r$ , their polarity (true or negated atom) is assigned randomly, and variables are instantiated with values sampled from the pool (line 16). Facts are created randomly, as we are not interested in teaching the model specific facts to recall later, but instead we want to teach it how to reason with different combinations of rules and facts. We then ensure that the rule is triggered in every context, eventually adding more facts to  $F$  using the function  $GetRuleFacts$  in line 17. After obtaining  $F$ , we feed rule  $r$  along with facts  $F$  to the  $LP^{MLN}$  reasoner, and we obtain a set  $O$  containing all satisfied facts and rule conclusions (line 4).

We generate different hypotheses, where each one leads to an example in dataset  $D$ . For each context, we add an example with different facts with respect to the given rule according to three dimensions. A fact can be (i) for a predicate in the premise or in the conclusion of a rule, could be (ii) satisfied or unsatisfied given the rule, and could have (iii) positive or negative polarity. This makes eight different possibilities, thus leading to the generation of eight different hypotheses (one for each context).

The first hypothesis  $h_1$  is obtained by sampling a fact from the set  $F$  (line 5). We then produce the counter hypothesis  $h_2$  by altering the fact (line 6) using the function  $Alter$  (lines 19–22). Given a hypothesis  $p(s, o)$  (line 19), we return its negated form if  $p$  is symmetric (line 20). Otherwise, if  $p$  is not symmetric, we produce a counter hypothesis either by negation (line 21), or by switching the subject and the object in the triple as the predicate is not symmetric (line 22). We rely on a dictionary to check whether a predicate is symmetric or not.

We then produce hypothesis  $h_3$  (line 7), which is the outcome of triggering rule  $r$  with the facts added in line 17. The counter hypothesis  $h_4$  is generated by altering  $h_3$  (line 8). Moreover, we generate hypothesis  $h_5$  by considering any unsatisfied positive fact outside  $F$ . Following a closed-world assumption (CWA), we assume that positive triples are false if they cannot be proven, meaning that their negation is true. We sample a fact  $f_l$  from the set of all possible positive facts that do not have the same predicate of the rule head (line 9). Thus,  $h_5$  will never be in the output  $O$  of the reasoner, as it cannot be derived. We then produce  $h_6$  by negating  $h_5$  in line 10. We further derive  $h_7$  by sampling a fact  $f_r$  that has the same predicate as that of the rule head, but does not belong to the output of the reasoner  $O$  (line 11). For a positive (negative) rule, such a fact is labelled as False (True).  $h_7$  is then negated to get the counter hypothesis  $h_8$  (line 12). All generated hypotheses are added to  $D$  (line 13), and the process repeats until we obtain  $n$  examples.

Finally, we automatically convert the examples to natural language using predefined templates. A basic template for atom predicate  $p$  (type  $t_1$ , type  $t_2$ ) is “If the 1<sup>st</sup>  $t_1$  is  $p$  of the 2<sup>nd</sup>  $t_2$ .” (“If the first person is spouse of . . .”). For the single-rule scenario, we release a dataset for 161 rules with a total of 3.2M examples and a 80%:10%:10% split for training, validation, and testing.

### 4.3 Rules with Overlapping Conclusion

When multiple rules are in the context, there could be facts that trigger more than one rule for a given hypothesis. The triggered rules might be all of the same polarity (positive or negative), eventually accumulating their confidence, or could be a mix of positive and negative rules that oppose each other. While the data generation procedure in Section 4.2 can be extended to handle multiple rules, this raises an efficiency problem. Given a set of  $R$  rules, it would generate  $8^{|R|}$  examples for each  $(facts, rule)$  pair in order to cover all rule combinations. This is very expensive, e.g., for five rules, it would generate  $8^5 = 32,768$  examples for a single context.

Given this challenge, we follow a different approach. We first generate data for each rule individually using Algorithm 1. We then generate more examples only for combinations of two or more rules having *all their rule conclusions* as hypotheses. For every input context, we produce rule-conclusion hypotheses (positive and negative) while varying the rules being fired. Thus, we generate  $2 * \sum_{x=2}^{|R|} \binom{|R|}{x}$  examples with at least two rules triggered. Adding the single-rule data, we generate  $8 * |R| + 2 * \sum_{x=2}^{|R|} \binom{|R|}{x}$  for every  $(facts, rules)$  pair, which is considerably smaller than  $8^r$  for  $|R| \geq 2$ , according to the binomial theorem. For example, for  $|R|=5$ , we generate 92 examples per context. For the overlapping rules scenario, we release a dataset for 5 rules with a total of 300K examples, and a 70%:10%:20% split for training, validation, and testing.

### 4.4 Chaining of Rule Executions

For certain hypotheses, an answer may be obtained by executing rules in a sequence, i.e., one on the result of the other, or in a *chain*. To be able to evaluate a model in this scenario, we generate hypotheses that can be tested only by chaining a number of rules (an example is shown in Appendix H). Given a pool of rules over different relations and a depth  $D$ , we sample a chain of rules with length  $D$ . We then generate hypotheses that would require a depth varying between 0 and  $D$ . We generate a rule-conclusion hypothesis ( $h_3$ ) and its alteration ( $h_4$ ) for each depth  $d \leq D$ . A depth of 0 means that the hypothesis can be verified using the facts alone without triggering any rule. We also generate counter-hypotheses by altering the hypotheses at a given depth, and we further include hypotheses that are unsatisfied given the input.

For the chaining rules scenario, we start with a pool of 64 soft rules, and we generate hypotheses that would need at most five chained rules to verify them. The dataset for  $d \leq 5$  contains a total of 70K examples, and a 70%:10%:20% split for training, validation, and testing.

## 5 Teaching PLMs to Reason

In this section, we explain how we teach a PLM to reason with one or more soft rules. Note that uncertainty stems from the rule confidence. One approach to teach how to estimate the probability of a prediction is to treat each confidence value (or bucket of confidence values) as a class and to model the problem as a  $k$ -way classification instance (or regression), but this is intractable when multiple rules are considered. Instead, we keep the problem as a two-class one by altering how the information is propagated in the model to incorporate uncertainty from the rule confidence.

Let  $D = \{(x_i, y_i)\}_{i=1}^m$  be our generated dataset, where  $x_i$  is one example of the form  $(context, hypothesis, confidence)$  and  $y_i$  is a label indicating whether the hypothesis is validated or not by the context (facts and rules in English), and  $m$  is the size of the training set. A classifier  $f$  is a function that maps the input to one of the labels in the label space. Let  $h(x, y)$  be a classification loss function. The empirical risk of the classifier  $f$  is

$$R_h(f) = \mathbb{E}_D(h(x, y)) = -\frac{1}{m} \sum_{i=1}^m h(x_i, y_i)$$

We want to introduce uncertainty in our loss function, using the weights computed by the  $LP^{MLN}$  solver as a proxy to represent the probability of predicting the hypothesis as being true. To do so, we apply a revised empirical risk:

$$R'_h(f) = \mathbb{E}_D(h(x, y)) = -\frac{1}{m} \sum_{i=1}^m (w(x_i) * h(x_i, 1) + (1 - w(x_i)) * h(x_i, 0))$$

where  $w(x_i)$  is the probability of  $x_i$  being True.

We now state that each example is considered as a combination both of a weighted positive example with a weight  $w(x_i)$  provided by the  $LP^{MLN}$  solver and a weighted negative example with a weight  $1 - w(x_i)$ .

When trained to minimize this risk, the model learns to assign the weights to each output class, thus predicting the confidence for the true class when given the satisfied rule head as a hypothesis.

| Dataset                          | Total | Train | Dev  | Test |
|----------------------------------|-------|-------|------|------|
| Single Rule (Section 6.2)        | 20K   | 16K   | 2K   | 2K   |
| Overlap (Section 6.3)            | 300K  | 210K  | 30K  | 60K  |
| Chaining (Depth=5) (Section 6.4) | 70K   | 56K   | 4.6K | 9.4K |
| RULEBERT (Section 7)             | 3.2M  | 2.56M | .32M | .32M |

Table 1: Datasets for the experiments and their splits.

## 6 Experiments

We first describe the experimental setup (Section 6.1). We then evaluate the model on single (Section 6.2) and on multiple rules (Sections 6.3 and 6.4). We show that a PLM fine-tuned on soft rules, namely RULEBERT, makes accurate predictions for unseen rules (Section 6.5), and it is more consistent than existing models on three external datasets (Section 7). We report the values of the hyper-parameters, as well as the results for some ablation experiments in the Appendix. The datasets for all experiments are summarized in Table 1.

### 6.1 Experimental Setup

**Rules.** We use a corpus of 161 soft rules mined from DBpedia. We chose a pool of distinct rules with varying number of variables, number of predicates, rule conclusions, and confidences.

**Reasoner.** We use the official implementation<sup>1</sup> of the  $LP^{MLN}$  reasoner. We set the reasoner to compute the exact probabilities for the triples.

**PLM.** We use the HuggingFace pre-trained *RoBERTa<sub>LARGE</sub>* (Liu et al., 2020) model as our base model, as it is trained on more data compared to *BERT* (Devlin et al., 2019), and is better at learning positional embeddings (Wang and Chen, 2020). We fine-tune the PLM<sup>2</sup> with the weighted binary cross-entropy (wBCE) loss from Section 5. More details can be found in Appendix C.

**Evaluations Measures.** For the examples in the test set, we use accuracy (Acc) and F1 score (F1) for balanced and unbalanced settings, respectively. As these measures do not take into account the uncertainty of the prediction *probability*, we further introduce Confidence Accuracy@k (CA@k), which measures the proportion of examples whose absolute error between the predicted and the actual probabilities is less than a threshold  $k$ :

$$CA@k = \frac{\#\{x_i, |w_i - \hat{w}_i| < k\}}{\#\{x_i\}}$$

<sup>1</sup><http://github.com/azreasoners/lpmln>

<sup>2</sup>The prompt is `<s>context</s></s>hypothesis</s>`.

where  $x_i$  is the  $i^{th}$  example of dataset,  $w_i$  is the actual confidence of the associated hypothesis given by the  $LP^{MLN}$  reasoner,  $\hat{w}_i$  is the predicted confidence by the model, and  $k$  is a chosen threshold.

The measure can be seen as the ordinary accuracy measure, but true positives and negatives are counted only if the condition is satisfied, where lower values for  $k$  indicate stricter evaluation.

### 6.2 Single Soft Rule

We fine-tune 16 models for 16 different positive and negative rules (one model per rule) using 16k training samples per rule. We compare the accuracy of each model (i) without teaching uncertainty using binary cross-entropy (RoBERTa), and (ii) with teaching soft rules using wBCE.

**Results.** Every row in Table 2 shows a rule with its confidence, followed by accuracy and CA@ $k$  (for  $k = 0.1$  and  $k = 0.01$ ) for both loss functions. We see that models fine-tuned using *RoBERTa-wBCE* perform better on CA@ $k$ . In terms of *accuracy*, both models perform well, with *RoBERTa-wBCE* performing better for all rules. Interestingly, the best performing rules are two rules that involve comparison of numerical values (birth years against death and founding years), which suggests that our method can handle comparison predicates.

### 6.3 Rules Overlapping on Conclusion

The dataset contains five soft rules with *spouse* or *negspouse* in the rule conclusion, and a confidence between 0.30 and 0.87 (shown in Figure 2). We train a model on the dataset and test it (i) on a test set for each of the five rules separately, (ii) on test sets with  $U$  triggered rules, where  $U \in \{2, 3, 4, 5\}$ .

|              |                                                                              |
|--------------|------------------------------------------------------------------------------|
| $(r_1, .87)$ | $\text{child}(a,c) \wedge \text{parent}(c,b) \rightarrow \text{spouse}(A,B)$ |
| $(r_2, .64)$ | $\text{child}(a,b) \rightarrow \text{negspouse}(a,b)$                        |
| $(r_3, .3)$  | $\text{relative}(a,b) \rightarrow \text{spouse}(a,b)$                        |
| $(r_4, .78)$ | $\text{child}(a,c) \wedge \text{child}(b,c) \rightarrow \text{spouse}(a,b)$  |
| $(r_5, .67)$ | $\text{predecessor}(a,b) \rightarrow \text{negspouse}(a,b)$                  |

Figure 2: The five overlapping soft rules.

**Results.** Table 3 shows that the model achieves high scores both on the single test sets (top five rows) and on the sets with interacting rules. The test sets with  $U = 2$  and  $U = 3$  are most challenging, as they contain  $\binom{5}{2} = 10$  and  $\binom{5}{3} = 10$  combinations of rules, respectively, while the one with  $U = 5$  has only one possible rule combination. The high scores indicate that PLMs can actually learn the interaction between multiple soft rules.